

Министерство образования Республики Беларусь
Учреждение образования
«Белорусский государственный университет информатики
и радиоэлектроники»

Кафедра программного обеспечения информационных технологий

Специальность «Программная инженерия»

Учебная дисциплина «Метрология, стандартизация и сертификация в
информационных технологиях»

ОТЧЕТ

по практической работе No1
«Метрики размера программ»

Подготовили:
Ст.гр.410902

Гавлев Н.В.
Джалилова Н.Р.

Проверил:

Болтак С.В.

Минск 2026

Язык для анализа: Rust

Задание: разработать программу, содержащую основные операторы языка и пользовательские подпрограммы, а также разработать программу-парсер для анализа исходного кода программы (парсер должен определять операторы и операнды программы и на их основе рассчитывать 6 базовых и 3 расширенные метрики Холстеда). Результаты анализа представить в виде таблицы с базовыми метриками и отдельным выводом расширенных метрик.

Правила, с учетом которых проводились расчеты:

Объявление переменных, функций, структур, перечислений, трейтов, реализаций, модулей и подключение библиотек само по себе не выделяется как отдельный операнд или оператор, за исключением тех ключевых слов и конструкций, которые парсер непосредственно классифицирует как операторы. Объявления функций не учитываются как вызовы этих функций.

Переменные и другие идентификаторы при их использовании считаются операндами.

Имена функций и методов считаются операторами.

Имена переменных, параметров, полей и других идентификаторов считаются операндами.

Совпадающие идентификаторы считаются одним и тем же операндом независимо от места их использования.

Стандартные типы Rust (i32, i64, u32, usize, f64, bool, char, str, String, Vec, Option, Result, Box и т. п.) не считаются ни операторами, ни операндами.

Пользовательские имена типов (struct, enum, trait) не считаются ни операторами, ни операндами в месте объявления.

Ключевые слова и служебные конструкции Rust из заданного списка считаются операторами. К ним относятся: if, else, match, loop, while, for, return, break, continue, await, yield, as.

Конструкция if ... else if ... else считается одним оператором if. Вложенные else if не считаются отдельными операторами. Ключевое слово else не считается отдельным оператором.

Циклы for, while, loop и конструкция match считаются одним оператором каждая. Ключевое слово in в конструкции for ... in ... не считается отдельным оператором.

Ключевые слова `true`, `false`, `None`, `Some`, `Ok`, `Err` считаются операндами.

Строковые, символьные и числовые литералы считаются операндами. Значение литерала используется как его представление в словаре операндов.

Круглые, квадратные и фигурные скобки при группировке подвыражения считаются одним оператором (пара скобок = 1 оператор).

Скобки при вызове функции считаются частью оператора-функции, а не отдельным оператором. Имя функции вместе со скобками считается одним оператором. Например: `sum_scores(scores)` — 1 оператор, `scores.len()` — 1 оператор, `println!("{}", ...)` — 1 оператор.

Все основные арифметические, логические, сравнительные, побитовые и составные операторы Rust считаются операторами: `+`, `-`, `*`, `/`, `%`, `==`, `!=`, `<`, `>`, `<=`, `>=`, `&&`, `||`, `!`, `&`, `|`, `^`, `<<`, `>>`, `=`, `+=`, `-=`, `*=`, `/=`, `%=`, `&=`, `|=`, `^=`, `<<=`, `>>=`.

Специальные символы Rust являются операторами: `?`, `->`, `=>`, `...`, `..=`, `@`, `::`, `.`

Разделители `;`, `,`, `:` считаются операторами.

Оператор `?` в конструкциях `Option` и `Result` считается оператором (распространение ошибки).

Угловые скобки `<` и `>` в дженериках (например, `Vec<i32>`) считаются операторами.

Срезы и индексация анализируются по общим правилам: `scores[i]` — `scores` операнд, `[]` оператор, `i` операнд. `&scores[i]` — `&` оператор, `scores` операнд, `[]` оператор, `i` операнд.

Форматная строка в макросе считается одним операндом целиком. Внутреннее содержимое `{}` не анализируется отдельно. Если в строке несколько `{}`, вся строка всё равно считается одним операндом.

Оператор `as` считается оператором (приведение типа). Тип после `as` не считается ни оператором, ни операндом.

Ключевые слова `fn`, `let`, `mut`, `const`, `static`, `extern`, `unsafe`, `async`, `move`, `ref`, `impl`, `struct`, `enum`, `trait`, `type`, `use`, `mod`, `where`, `pub`, `crate`, `super`, `Self`, `dyn` не считаются ни операторами, ни операндами.

Переменная `self` считается операндом. Имя типа `Self` считается операндом.

Комментарии, атрибуты (`#[...]`), лайфтаймы (`'a', 'static'`) и метки (`'label:'`) не учитываются при расчёте метрик.

Пробелы, табуляции и переводы строк не являются ни операторами, ни операндами.

Регистр идентификаторов имеет значение. Например, `Total` и `total` рассматриваются как разные операнды.

Листинг анализируемого кода:

```
fn sum_scores(scores: &[i32]) -> i32 {
    let mut total = 0;
    for s in scores {
        total = total + *s;
    }
    return total;
}
fn average(scores: &[i32]) -> f64 {
    let total = sum_scores(scores);
    let n = scores.len() as f64;
    return total as f64 / n;
}
fn maximum(scores: &[i32]) -> i32 {
    let mut best = scores[0];
    for s in scores {
        if *s > best {
            best = *s;
        }
    }
    return best;
}
fn minimum(scores: &[i32]) -> i32 {
    let mut worst = scores[0];
    for s in scores {
        if *s < worst {
            worst = *s;
        }
    }
    return worst;
}
fn count_fail(scores: &[i32]) -> i32 {
    let mut n = 0;
    for s in scores {
        if *s <= 4 {
```

```

        n = n + 1;
    }
}
return n;
}
fn scholarship(avg: f64) -> f64 {
    if avg >= 9.0 {
        return 321.65;
    } else if avg >= 8.0 {
        return 281.44;
    } else if avg >= 6.0 {
        return 241.24;
    } else if avg >= 5.0 {
        return 201.03;
    } else {
        return 0.0;
    }
}
}
fn main() {
    let names = ["Анна", "Борис", "Виктор", "Галина", "Дмитрий"];
    let scores = [
        [10, 9, 10, 9, 10],
        [5, 4, 6, 5, 4],
        [7, 8, 7, 6, 8],
        [9, 10, 9, 10, 9],
        [3, 4, 2, 5, 4],
    ];
    let mut best_avg = 0.0;
    let mut best_index = 0;
    let mut i = 0;
    while i < names.len() {
        let name = names[i];
        let row = &scores[i];
        let total = sum_scores(row);
        let avg = average(row);
        let best = maximum(row);
        let worst = minimum(row);
        let fails = count_fail(row);
        let money = scholarship(avg);
        println!("=== {} ===", name);
        println!("Оценки: {:?}", row);
        println!("Сумма баллов: {}", total);
        println!("Средний балл: {:.2}", avg);
        println!("Лучшая оценка: {}", best);
        println!("Худшая оценка: {}", worst);
    }
}

```

```

println!("Неудов: {}", fails);
println!("Стипендия: {:.2} руб.", money);
if avg > best_avg {
    best_avg = avg;
    best_index = i;
}
i = i + 1;
}
println!("--- Итог группы ---");
println!("Лучший студент: {}", names[best_index]);
println!("Средний балл: {:.2}", best_avg);
}

```

Результаты расчета базовых метрик Холстеда, посчитанных вручную, приведены в таблице 1.

Таблица 1– Расчет базовых метрик Холстеда (ручной подсчет)

j	Оператор	f1j	i	Операнд	f2i
1.	;	47	1.	scores	14
2.	sum_scores()	2	2.	total	8
3.	:	6	3.	0	6
4.	&	6	4.	s	10
5.	[]	17	5.	n	6
6.	->	6	6.	best	6
7.	=	26	7.	worst	6
8.	+	3	8.	4	5
9.	*	6	9.	1	2
10.	for	4	10.	9.0	1
11.	return	10	11.	8.0	1
12.	scores.len()	1	12.	6.0	1
13.	as	2	13.	5.0	1
14.	/	1	14.	321.65	1
15.	maximum()	1	15.	281.44	1
16.	minimum()	1	16.	241.24	1
17.	average()	1	17.	avg	10
18.	>	2	18.	0.0	2
19.	<	2	19.	201.03	1
20.	count_fail()	1	20.	10	5
21.	<=	1	21.	6	2
22.	>=	4	22.	9	5
23.	if	5	23.	5	3
24.	while	1	24.	7	2

25.	names.len()	1	25.	8	2
26.	println!()	11	26.	3	1
27.	scholarship()	1	27.	2	1
28.	,	39	28.	best_avg	4
			29.	best_index	3
			30.	i	7
			31.	names	3
			32.	row	7
			33.	fails	2
			34.	money	2
			35.	"--- Итог группы - --"	1
			36.	"Лучший студент: {}"	1
			37.	"Средний балл: {:.2}"	2
			38.	"Стипендия: {:.2} руб."	1
			39.	"Худшая оценка: {}"	1
			40.	"Неудов: {}"	1
			41.	"Лучшая оценка: {}"	1
			42.	"Сумма баллов: {}"	1
			43.	"Оценки: {:?}"	1
			44.	"Анна"	1
			45.	"Борис"	1
			46.	"Виктор"	1
			47.	"Галина"	1
			48.	"Дмитрий"	1
			49.	"=== {} ==="	1
			50.	name	2
η1 = 28		N1 = 208	η2 = 50		N2 = 150

Словарь программы $\eta = 28 + 50 = 78$.

Длина программы $N = 208 + 150 = 358$.

Объем программы $V = 358 \log_2 (78) = 2250$

Скрин с результатами работы парсера (расчет базовых метрик Холстеда в таблицах и вывод расширенных метрик Холстеда) представлен на рисунках 1, 2, 3, 4, 5.

$\eta^1 = 28$		$N^1 = 208$

Рисунок 2 – Таблица операторов, расчет базовых метрик Холстеда (парсер)

i	Операнд	f2i
1	scores	14
2	avg	10
3	s	10
4	total	8
5	i	7
6	row	7
7	0	6
8	best	6
9	n	6
10	worst	6
11	10	5
12	4	5
13	9	5
14	best_avg	4
15	5	3
16	best_index	3
17	names	3
18	"Средний балл: {:.2}"	2
19	0.0	2
20	1	2
21	6	2
22	7	2
23	8	2
24	fails	2
25	money	2
26	name	2
27	"--- Итог группы ---"	1
28	"=== {} ==="	1
29	"Анна"	1
30	"Борис"	1
31	"Виктор"	1
32	"Галина"	1
33	"Дмитрий"	1
34	"Лучшая оценка: {}"	1
35	"Лучший студент: {}"	1
36	"Неудов: {}"	1
37	"Оценки: {}?"	1
38	"Стипендия: {:.2} руб."	1
39	"Сумма баллов: {}"	1
40	"Худшая оценка: {}"	1
41	2	1
42	201.03	1
43	241.24	1
44	281.44	1

Рисунок 3 – Таблица операндов, расчет базовых метрик Холстеда (парсер)

45	3	1
46	321.65	1
47	5.0	1
48	6.0	1
49	8.0	1
50	9.0	1
$\eta^2 = 50$		N2 = 150

Рисунок 4 – Таблица операндов, расчет базовых метрик Холстеда (парсер)

Словарь программы: $\eta = \eta_1 + \eta_2 = 28 + 50 = 78$

Длина программы: $N = N_1 + N_2 = 208 + 150 = 358$

Объем программы: $V = N \cdot \log_2 \eta = 358 \cdot \log_2 78 = 2250$

[← Назад](#)

Рисунок 5 – Расчет расширенных метрик Холстеда (парсер)

Вывод: в ходе работы была разработана программа на языке Rust, содержащая основные операторы языка и пользовательские функции, а также программа-парсер для анализа исходного кода. На основе результатов анализа были рассчитаны базовые и расширенные метрики Холстеда.